```
import json
from arithmetic import (alu_add_sub, carry_lookahead_adder, booth_multiply,
                        restoring_division, non_restoring_division, ieee754_add)
from pipeline import sample_program, run_pipeline, performance_metrics,
    superscalar_and_smt_comparison


def demo_alu():
    print("=== Integer ALU: Add / Subtract (two's complement, 8-bit) ===")
    for a, b in [(45, 30), (-45, 30), (100, 50)]:
        r, c, ov = alu_add_sub(a, b, subtract=False)
        print(f"  {a:4d} + {b:4d} = {r:4d}   carry={c} overflow={ov}")
    for a, b in [(45, 30), (30, 45), (-20, 50)]:
        r, c, ov = alu_add_sub(a, b, subtract=True)
        print(f"  {a:4d} - {b:4d} = {r:4d}   carry={c} overflow={ov}")
    return True


def demo_cla():
    print("\n=== Carry Look-Ahead Adder ===")
    results = []
    for a, b in [(77, 45), (-10, 25)]:
        r, cout, trace = carry_lookahead_adder(a, b)
        print(f"  {a} + {b} = {r}  (carry_out={cout})")
        print(f"    G={trace['G']}")
        print(f"    P={trace['P']}")
        print(f"    carry_chain={trace['carry_chain']}")
        results.append((a, b, r))
    return results
```

```
def demo_booth():
    print("\n=== Booth's Multiplication ===")
    cases = [(13, -6), (-9, -12), (23, 11)]
    for m, r in cases:
        product, trace = booth_multiply(m, r)
        expected = m * r
        status = "OK" if product == expected else "MISMATCH"
        print(f"  {m} x {r} = {product}  (expected {expected}) [{status}], steps={len(trace)}")
    return cases


def demo_division():
    print("\n=== Restoring vs Non-Restoring Division (unsigned, 8-bit) ===")
    cases = [(23, 5), (100, 7), (200, 13)]
    for dividend, divisor in cases:
        q1, r1, _ = restoring_division(dividend, divisor)
        q2, r2, _ = non_restoring_division(dividend, divisor)
        py_q, py_r = divmod(dividend, divisor)
        print(f"  {dividend} / {divisor} -> restoring Q={q1} R={r1} | "
              f"non-restoring Q={q2} R={r2} | python Q={py_q} R={py_r}")
    return cases


def demo_ieee754():
    print("\n=== IEEE 754 Single-Precision Addition ===")
    cases = [(6.25, 3.5), (0.1, 0.2), (-12.75, 4.5)]
    for x, y in cases:
        result, trace = ieee754_add(x, y)
        print(f"  {x} + {y} = {result}  (python float add = {x + y})")
        print(f"    sx={trace.get('sx')} ex={trace.get('ex')} mx={trace.get('mx')} | "
              f"sy={trace.get('sy')} ey={trace.get('ey')} my={trace.get('my')}")
    return cases
```

```
def demo_pipeline():
```

```
    print("\n=== 5-Stage Pipeline: Hazard Handling Comparison ===")
    program = sample_program()

    result_no_fw = run_pipeline(program, forwarding=False)
    result_fw = run_pipeline(program, forwarding=True)

    print(f"  Program ({len(program)} instructions):")
    for line in result_fw["program"]:
        print(f"    {line}")

    print(f"\n  Stall-only (no forwarding): total_cycles={result_no_fw['total_cycles']}, "
          f"stalls={result_no_fw['total_stalls']}, CPI={result_no_fw['cpi']}")
    print(f"  With forwarding:           total_cycles={result_fw['total_cycles']}, "
          f"stalls={result_fw['total_stalls']} "
          f"(data={result_fw['data_hazard_stalls']}, control={result_fw['control_hazard_flushes']}), "
          f"CPI={result_fw['cpi']}")

    perf_no_fw = performance_metrics(result_no_fw["total_cycles"], clock_period_ns=1.0,
      n_instructions=len(program))
    perf_fw = performance_metrics(result_fw["total_cycles"], clock_period_ns=1.0,
      n_instructions=len(program))
    print(f"\n  Performance @1GHz (1ns cycle):")
    print(f"    No forwarding : CPI={perf_no_fw['cpi']}, exec_time={perf_no_fw['execution_time_ns']}ns, "
          f"throughput={perf_no_fw['throughput_mips']} MIPS")
    print(f"    Forwarding    : CPI={perf_fw['cpi']}, exec_time={perf_fw['execution_time_ns']}ns, "
          f"throughput={perf_fw['throughput_mips']} MIPS")
```

```
def demo_superscalar():
    print("\n=== Superscalar / SMT Analytical Comparison (200-instruction benchmark) ===")
    comp = superscalar_and_smt_comparison()
    for label, data in comp.items():
        print(f"  {label:11s}: cycles={data['cycles']:>7} CPI={data['cpi']:.3f} IPC={data['ipc']:.3f}")
    speedup_super = comp["scalar"]["cycles"] / comp["superscalar"]["cycles"]
```

```
    speedup_smt = comp["scalar"]["cycles"] / comp["smt"]["cycles"] * 2  # SMT does 2x the work in fewer
        cycles
    print(f"  Speedup, scalar -> superscalar: {speedup_super:.2f}x")
    print(f"  Effective speedup, scalar -> SMT (2 threads): {speedup_smt:.2f}x")
    return comp


def main():
    results = {}
    results["alu"] = demo_alu()
    results["cla"] = demo_cla()
    results["booth"] = demo_booth()
    results["division"] = demo_division()
    results["ieee754"] = demo_ieee754()
    results["pipeline"] = demo_pipeline()
    results["superscalar"] = demo_superscalar()
    print("\nAll functional units executed successfully.")
    return results
```

```
if __name__ == "__main__":
    main()

"""
pipeline.py
Five-stage pipelined datapath (IF ID EX MEM WB), hazard detection/handling,
and a simplified superscalar / SMT comparison used for the performance study.
"""

STAGES = ["IF", "ID", "EX", "MEM", "WB"]


class Instruction:
    def __init__(self, text, op, dest=None, src1=None, src2=None, is_branch=False, is_load=False):
        self.text = text
        self.op = op
        self.dest = dest
```

```
        self.src1 = src1
        self.src2 = src2
        self.is_branch = is_branch
        self.is_load = is_load
```

```
def sample_program():
    """A small instruction stream that deliberately contains data and control hazards."""
    return [
        Instruction("LOAD  R1, 0(R0)",  "LOAD", dest="R1", src1="R0", is_load=True),
        Instruction("ADD   R2, R1, R1", "ADD",  dest="R2", src1="R1", src2="R1"),   # RAW on R1
        Instruction("SUB   R3, R2, R1", "SUB",  dest="R3", src1="R2", src2="R1"),   # RAW on R2
        Instruction("BEQ   R3, R0, L1", "BEQ",  src1="R3", src2="R0", is_branch=True),
        Instruction("ADD   R4, R3, R3", "ADD",  dest="R4", src1="R3", src2="R3"),
        Instruction("STORE R4, 0(R0)",  "STORE", src1="R4"),
    ]


def _producer_of(program, idx, reg):
    """Nearest earlier instruction (if any) that writes `reg`."""
    for p in range(idx - 1, -1, -1):
        if program[p].dest == reg:
            return p
    return None


def run_pipeline(program, forwarding=True, branch_penalty=2):
    """
    Cycle-by-cycle model of a 5-stage in-order scalar pipeline (single issue).
    One instruction is fetched per cycle unless a hazard holds it in ID.
```

```
n = len(program)
stage_idx = [-1] * n        # -1 not issued, 0..4 = IF..WB, 5 = retired
cycle_log = [[] for _ in range(n)]
stall_cycles = [0] * n
branch_flushes = 0
next_to_issue = 0
cyc = 0
branch_resolved_at = {}     # idx -> cycle EX completes, for flush accounting

max_cycles = n * 8 + 20
while cyc < max_cycles and not all(s == 5 for s in stage_idx):
    cyc += 1

    held = [False] * n
    # advance back-to-front so a freed stage isn't reused the same cycle
    for idx in range(n - 1, -1, -1):
        if 0 <= stage_idx[idx] < 4:
            blocked = False
            if stage_idx[idx] == 1:  # sitting in ID: check RAW hazard
                instr = program[idx]
                for r in (instr.src1, instr.src2):
                    if not r:
                        continue
                    producer = _producer_of(program, idx, r)
                    if producer is None or stage_idx[producer] == 5:
                        continue
                    p_stage = stage_idx[producer]
                    if forwarding:
                        if program[producer].is_load and p_stage <= 2:
                            blocked = True
                    else:
                        if p_stage < 4:
```

```
                    blocked = True
            if blocked:
                stall_cycles[idx] += 1
                held[idx] = True
            else:
                stage_idx[idx] += 1

    if next_to_issue < n and not any(stage_idx[j] == 0 for j in range(n)):
        stage_idx[next_to_issue] = 0
        next_to_issue += 1

    for idx in range(n):
        if program[idx].is_branch and stage_idx[idx] == 2 and idx not in branch_resolved_at:
            branch_resolved_at[idx] = cyc
            branch_flushes += branch_penalty
            next_to_issue = max(next_to_issue, idx + 1)

    for idx in range(n):
        if 0 <= stage_idx[idx] <= 4 and not held[idx]:
            cycle_log[idx].append((cyc, STAGES[stage_idx[idx]]))
        elif held[idx]:
            cycle_log[idx].append((cyc, "stall"))
        if stage_idx[idx] == 4:
            stage_idx[idx] = 5

total_cycles = cyc
total_stalls = sum(stall_cycles) + branch_flushes
cpi = round(total_cycles / n, 3)
return {
    "program": [ins.text for ins in program],
    "cycle_log": cycle_log,
    "total_cycles": total_cycles,
    "data_hazard_stalls": sum(stall_cycles),
    "control_hazard_flushes": branch_flushes,
    "total_stalls": total_stalls,
    "cpi": cpi,
```

```
def performance_metrics(total_cycles, clock_period_ns, n_instructions):
    """CPI, execution time and throughput (MIPS) from a pipeline run."""
    cpi = total_cycles / n_instructions
    exec_time_ns = total_cycles * clock_period_ns
    throughput_mips = (n_instructions / (exec_time_ns * 1e-9)) / 1e6
    return {
        "cpi": round(cpi, 3),
        "execution_time_ns": round(exec_time_ns, 2),
        "throughput_mips": round(throughput_mips, 3),
    }


def superscalar_and_smt_comparison(n_instructions=200, cycles_scalar_per_instr=1.35,
                                   issue_width=2, superscalar_ipc_efficiency=0.8,
                                   smt_threads=2, smt_efficiency=0.65):
    """
    Simplified analytical comparison (not cycle-accurate) of:
      - single-issue in-order scalar pipeline
      - N-wide superscalar with out-of-order + speculative execution
      - simultaneous multithreading (SMT) on top of the superscalar core
    Efficiency factors model the fact real IPC never scales linearly with
    issue width because of dependencies and branch mispredictions.
    """
    scalar_cycles = n_instructions * cycles_scalar_per_instr
    scalar_cpi = scalar_cycles / n_instructions

    superscalar_ipc = issue_width * superscalar_ipc_efficiency
    superscalar_cycles = n_instructions / superscalar_ipc
    superscalar_cpi = superscalar_cycles / n_instructions

    smt_ipc = superscalar_ipc * (1 + (smt_threads - 1) * smt_efficiency)
    smt_cycles = (n_instructions * smt_threads) / smt_ipc
    smt_cpi = smt_cycles / (n_instructions * smt_threads)
```

```
    return {
        "scalar":     {"cycles": round(scalar_cycles, 1), "cpi": round(scalar_cpi, 3), "ipc": round(1 / scalar_cpi,
        3)},
        "superscalar": {"cycles": round(superscalar_cycles, 1), "cpi": round(superscalar_cpi, 3), "ipc":
        round(superscalar_ipc, 3)},
        "smt":        {"cycles": round(smt_cycles, 1), "cpi": round(smt_cpi, 3), "ipc": round(smt_ipc, 3),
                      "threads": smt_threads},
    }
```

MAIN PY:

```
"""
main.py
Integrated Processor Architecture Simulator - entry point.
Each functional demo lives in its own method and is invoked from main(),
so every unit (ALU, CLA, Booth, division, IEEE 754, pipeline, superscalar)
can also be run/tested independently.
"""
import json
from arithmetic import (alu_add_sub, carry_lookahead_adder, booth_multiply,
                        restoring_division, non_restoring_division, ieee754_add)
from pipeline import sample_program, run_pipeline, performance_metrics,
    superscalar_and_smt_comparison


def demo_alu():
    print("=== Integer ALU: Add / Subtract (two's complement, 8-bit) ===")
    for a, b in [(45, 30), (-45, 30), (100, 50)]:
        r, c, ov = alu_add_sub(a, b, subtract=False)
        print(f"  {a:4d} + {b:4d} = {r:4d}   carry={c} overflow={ov}")
    for a, b in [(45, 30), (30, 45), (-20, 50)]:
        r, c, ov = alu_add_sub(a, b, subtract=True)
        print(f"  {a:4d} - {b:4d} = {r:4d}   carry={c} overflow={ov}")
    return True
```

```
def demo_cla():
    print("\n=== Carry Look-Ahead Adder ===")
```

```
    results = []
    for a, b in [(77, 45), (-10, 25)]:
        r, cout, trace = carry_lookahead_adder(a, b)
        print(f"  {a} + {b} = {r}  (carry_out={cout})")
        print(f"    G={trace['G']}")
        print(f"    P={trace['P']}")
        print(f"    carry_chain={trace['carry_chain']}")
        results.append((a, b, r))
    return results


def demo_booth():
    print("\n=== Booth's Multiplication ===")
    cases = [(13, -6), (-9, -12), (23, 11)]
    for m, r in cases:
        product, trace = booth_multiply(m, r)
        expected = m * r
        status = "OK" if product == expected else "MISMATCH"
        print(f"  {m} x {r} = {product}  (expected {expected}) [{status}], steps={len(trace)}")
    return cases
```

```
def demo_division():
    print("\n=== Restoring vs Non-Restoring Division (unsigned, 8-bit) ===")
    cases = [(23, 5), (100, 7), (200, 13)]
    for dividend, divisor in cases:
        q1, r1, _ = restoring_division(dividend, divisor)
        q2, r2, _ = non_restoring_division(dividend, divisor)
        py_q, py_r = divmod(dividend, divisor)
        print(f"  {dividend} / {divisor} -> restoring Q={q1} R={r1} | "
              f"non-restoring Q={q2} R={r2} | python Q={py_q} R={py_r}")
    return cases


def demo_ieee754():
    print("\n=== IEEE 754 Single-Precision Addition ===")
    cases = [(6.25, 3.5), (0.1, 0.2), (-12.75, 4.5)]
```

```
    for x, y in cases:
        result, trace = ieee754_add(x, y)
        print(f"  {x} + {y} = {result}  (python float add = {x + y})")
        print(f"    sx={trace.get('sx')} ex={trace.get('ex')} mx={trace.get('mx')} | "
              f"sy={trace.get('sy')} ey={trace.get('ey')} my={trace.get('my')}")
    return cases
```

```
def demo_pipeline():
    print("\n=== 5-Stage Pipeline: Hazard Handling Comparison ===")
    program = sample_program()

    result_no_fw = run_pipeline(program, forwarding=False)
    result_fw = run_pipeline(program, forwarding=True)

    print(f"  Program ({len(program)} instructions):")
    for line in result_fw["program"]:
        print(f"    {line}")

    print(f"\n  Stall-only (no forwarding): total_cycles={result_no_fw['total_cycles']}, "
          f"stalls={result_no_fw['total_stalls']}, CPI={result_no_fw['cpi']}")
    print(f"  With forwarding:          total_cycles={result_fw['total_cycles']}, "
          f"stalls={result_fw['total_stalls']} "
          f"(data={result_fw['data_hazard_stalls']}, control={result_fw['control_hazard_flushes']}), "
          f"CPI={result_fw['cpi']}")

    perf_no_fw = performance_metrics(result_no_fw["total_cycles"], clock_period_ns=1.0,
        n_instructions=len(program))
    perf_fw = performance_metrics(result_fw["total_cycles"], clock_period_ns=1.0,
        n_instructions=len(program))
    print(f"\n  Performance @1GHz (1ns cycle):")
    print(f"    No forwarding : CPI={perf_no_fw['cpi']}, exec_time={perf_no_fw['execution_time_ns']}ns, "
          f"throughput={perf_no_fw['throughput_mips']} MIPS")
    print(f"    Forwarding    : CPI={perf_fw['cpi']}, exec_time={perf_fw['execution_time_ns']}ns, "
          f"throughput={perf_fw['throughput_mips']} MIPS")
```

```
def demo_superscalar():
    print("\n=== Superscalar / SMT Analytical Comparison (200-instruction benchmark) ===")
    comp = superscalar_and_smt_comparison()
    for label, data in comp.items():
        print(f"  {label:11s}: cycles={data['cycles']:>7} CPI={data['cpi']:.3f} IPC={data['ipc']:.3f}")
    speedup_super = comp["scalar"]["cycles"] / comp["superscalar"]["cycles"]
    speedup_smt = comp["scalar"]["cycles"] / comp["smt"]["cycles"] * 2  # SMT does 2x the work in fewer
      cycles
    print(f"  Speedup, scalar -> superscalar: {speedup_super:.2f}x")
    print(f"  Effective speedup, scalar -> SMT (2 threads): {speedup_smt:.2f}x")
    return comp


def main():
    results = {}
    results["alu"] = demo_alu()
    results["cla"] = demo_cla()
    results["booth"] = demo_booth()
    results["division"] = demo_division()
    results["ieee754"] = demo_ieee754()
    results["pipeline"] = demo_pipeline()
    results["superscalar"] = demo_superscalar()
    print("\nAll functional units executed successfully.")
    return results
```